

# 30 Bài Tập Bảng Băm & Hàm Băm

## PHẦN A — BẢNG BĂM (HASH TABLE) · 15 bài

### Bài 1. Cài đặt bảng băm bằng chaining

Cài đặt bảng băm với **danh sách liên kết tại mỗi bucket** (separate chaining), hỗ trợ put, get, remove.

Ví dụ: `put('a',1); get('a') → 1`

### Bài 2. Cài đặt bảng băm bằng dò tuyến tính

Cài đặt bảng băm **địa chỉ mở** (open addressing) với **linear probing**: khi va chạm, thử ô kế tiếp.

Ví dụ: va chạm tại idx  $i \rightarrow$  thử  $i+1, i+2, \dots$

### Bài 3. Đếm tần suất

Dùng bảng băm đếm số lần xuất hiện của mỗi phần tử trong một mảng.

Ví dụ: `[a,b,a,c,a] → {a:3, b:1, c:1}`

### Bài 4. Hai mảng có phần tử chung

Cho hai mảng, kiểm tra (và liệt kê) các phần tử **xuất hiện ở cả hai**, dùng tập băm  $O(n)$ .

Ví dụ: `[1,2,3]` và `[2,3,4] → {2,3}`

### Bài 5. Nhóm theo khóa (group by)

Nhóm các phần tử theo một khóa dẫn xuất, dùng bảng băm ánh xạ khóa  $\rightarrow$  danh sách.

Ví dụ: nhóm từ theo chữ cái đầu

### Bài 6. So sánh chaining vs open addressing

Cài cả hai cách xử lý va chạm và so sánh: bộ nhớ, hiệu năng khi hệ số tải cao, và cách xử lý xóa phần tử.

Ví dụ: phân tích ưu/nhược của hai phương pháp

### Bài 7. Hệ số tải và rehashing

Khi **hệ số tải** (load factor  $= n/m$ ) vượt ngưỡng (vd 0.75), cấp phát bảng lớn hơn và **băm lại** toàn bộ phần tử. Cài đặt cơ chế này.

Ví dụ: `load > 0.75 →` tăng gấp đôi  $m$ , rehash

### Bài 8. Quadratic probing / double hashing

Cài đặt **quadratic probing** và **double hashing** để giảm hiện tượng gom cụm (clustering) so với linear probing.

Ví dụ: bước dò  $= h_2(k)$  thay vì 1

### Bài 9. Two Sum dùng hash

Tìm hai chỉ số có  $a[i] + a[j] = \text{target}$  trong  $O(n)$  bằng bảng băm lưu phần bù.

Ví dụ: `a=[2,7,11], target=9 → (0,1)`

## Bài 10. Phần tử không lặp đầu tiên

Tìm **phần tử đầu tiên không bị lặp** trong một dãy (ví dụ ký tự đầu tiên xuất hiện một lần) dùng bảng băm đếm.

Ví dụ: 'leetcode' → 'l'

## Bài 11. Cài đặt HashSet

Cài đặt tập hợp băm với add, contains, remove, đảm bảo không có phần tử trùng.

Ví dụ: add 1,1,2 → set {1,2}

## Bài 12. Tổng đoạn con bằng k

Đếm số **đoạn con liên tiếp** có tổng bằng k, dùng **tổng tiền tố + bảng băm** trong  $O(n)$ .

Ví dụ: a=[1,1,1], k=2 → 2 đoạn

## Bài 13. Dãy liên tiếp dài nhất

Tìm độ dài **dãy số nguyên liên tiếp dài nhất** trong một mảng (không cần kề nhau), trong  $O(n)$  dùng tập băm.

Ví dụ: [100,4,200,1,3,2] → 4 (dãy 1,2,3,4)

## Bài 14. Xóa lười trong open addressing

Cài đặt **xóa lười** (lazy deletion) bằng nhãn "đã xóa" (tombstone) để không phá vỡ chuỗi dò tìm. Thảo luận khi nào cần dọn dẹp lại bảng.

Ví dụ: đánh dấu DELETED thay vì xóa hẳn

## Bài 15. Băm nhất quán (Consistent Hashing)

Thiết kế **consistent hashing** để phân tán khóa cho nhiều máy chủ sao cho khi thêm/bớt một máy chủ, số khóa phải di chuyển là tối thiểu (dùng vòng băm và nút ảo).

Ví dụ: thêm 1 server → chỉ  $\sim 1/n$  khóa di chuyển

# PHẦN B — HÀM BĂM (HASH FUNCTION) · 15 bài

## Bài 1. Hàm băm modulo cho số nguyên

Cài đặt hàm băm đơn giản  $h(k) = k \bmod m$ . Tính chỉ số bucket cho một tập khóa và quan sát phân bố.

Ví dụ: k=37, m=10 → 7

## Bài 2. Hàm băm cho chuỗi (tổng mã ký tự)

Cài đặt hàm băm cho chuỗi bằng tổng mã ký tự rồi lấy mod m. Chỉ ra nhược điểm (các đảo chữ cho cùng giá trị).

Ví dụ: 'abc' và 'cba' → cùng hash

## Bài 3. Hàm băm đa thức (polynomial)

Cài đặt **polynomial rolling hash**:  $h = (s[0] \cdot p^{(n-1)} + \dots + s[n-1]) \bmod m$ . Giải thích vai trò của p và m.

Ví dụ: p=31, m lớn nguyên tố

## Bài 4. Đếm va chạm của một hàm băm

Cho một tập khóa và một hàm băm, **đếm số va chạm** (số cặp khóa cùng bucket) để đánh giá chất lượng phân bố.

Ví dụ: ít va chạm → hàm băm tốt

## Bài 5. Vì sao chọn $m$ là số nguyên tố

Giải thích và minh họa bằng thực nghiệm vì sao chọn kích thước bảng  $m$  là số nguyên tố (và tránh lũy thừa của 2) giúp phân bố đều hơn với một số hàm băm.

Ví dụ: so phân bố khi  $m=16$  vs  $m=17$

## Bài 6. Rolling hash & Rabin-Karp

Dùng **rolling hash** để tìm một chuỗi mẫu trong văn bản (thuật toán Rabin-Karp) trong  $O(n+m)$  trung bình, cập nhật hash cửa sổ trong  $O(1)$ .

Ví dụ: tìm 'abc' trong 'zabcd' → vị trí 1

## Bài 7. Hàm băm cho cặp / tuple

Thiết kế hàm băm cho cặp  $(a, b)$  hoặc tuple, kết hợp hash thành phần sao cho ít va chạm (ví dụ kiểu hash\_combine).

Ví dụ:  $h(a, b) = h(a) * C \wedge h(b)$

## Bài 8. So sánh chất lượng phân bố

So sánh hai hàm băm trên cùng tập khóa bằng cách đếm số phần tử mỗi bucket và tính độ lệch (ví dụ thống kê chi-square). Kết luận hàm nào đều hơn.

Ví dụ: phân bố đều → các bucket cân bằng

## Bài 9. Băm phổ quát (Universal Hashing)

Cài đặt một **họ hàm băm phổ quát**, ví dụ  $h(k) = ((a \cdot k + b) \bmod p) \bmod m$  với  $a, b$  ngẫu nhiên. Giải thích vì sao nó chống được đầu vào xấu cố ý.

Ví dụ: chọn  $a, b$  ngẫu nhiên mỗi lần khởi tạo

## Bài 10. Phương pháp nhân (multiplication method)

Cài đặt hàm băm theo **phương pháp nhân**:  $h(k) = \text{floor}(m \cdot (k \cdot A \bmod 1))$  với  $0 < A < 1$ . So sánh với phương pháp chia.

Ví dụ:  $A \approx 0.618$  (tỉ lệ vàng)

## Bài 11. Hàm băm độc lập thứ tự

Thiết kế hàm băm cho một **tập hợp** (hoặc multiset) sao cho kết quả **không phụ thuộc thứ tự** phần tử (ví dụ cộng/XOR các hash thành phần).

Ví dụ:  $\text{hash}(\{1, 2, 3\}) = \text{hash}(\{3, 1, 2\})$

## Bài 12. Tấn công hash flooding

Minh họa **tấn công hash flooding**: tạo nhiều khóa cùng bucket khiến bảng băm suy biến về  $O(n)$  mỗi thao tác. Trình bày cách phòng chống (băm ngẫu nhiên hóa có khóa bí mật).

Ví dụ: nhiều va chạm cố ý → bảng băm chậm  $O(n)$

## Bài 13. Rolling hash 2 chiều

Mở rộng rolling hash để tìm một **ma trận con** trong ma trận lớn (2D pattern matching), kết hợp hash theo hàng và theo cột.

Ví dụ: tìm khối  $p \times q$  trong ma trận  $m \times n$

## Bài 14. Bloom Filter

Cài đặt **Bloom filter** dùng **nhiều hàm băm** để kiểm tra thành viên xấp xỉ (không âm tính giả). Phân tích xác suất dương tính giả theo số hàm băm  $k$  và kích thước bit  $m$ .

Ví dụ: có thể báo 'có' nhầm, không bao giờ báo 'không' nhầm

## Bài 15. MinHash ước lượng độ tương đồng

Dùng **MinHash** (nhiều hàm băm) để ước lượng **độ tương đồng Jaccard** giữa hai tập hợp lớn mà không cần so sánh trực tiếp toàn bộ.

Ví dụ: xấp xỉ  $|A \cap B| / |A \cup B|$  bằng các min-hash